

DOCKER (OVERVIEW)

Page 1

1. WHAT IS DOCKER ?

Docker is a platform that helps us build, package and run applications inside containers.

It ensures consistency across different environments.

2. WHY DOCKER ?

- Consistent environment
- Easy deployment
- Lightweight (vs VMs)
- Faster startup
- Isolation & Security



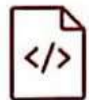
3. CORE CONCEPTS



Image → Read-only blueprint to create containers.

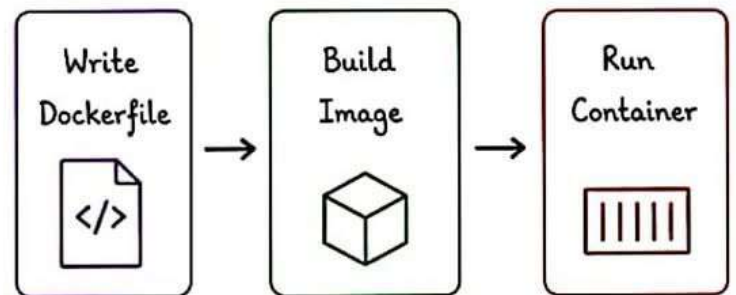


Container → Running instance of an image.



Dockerfile → Instructions to build an image.

4. BASIC FLOW



5. COMMON COMMANDS

<code>docker build -t myapp .</code>	→ Build an image from Dockerfile
<code>docker run -d myapp</code>	→ Run a container from image
<code>docker ps</code>	→ List running containers
<code>docker ps -a</code>	→ List all containers
<code>docker stop <container_id></code>	→ Stop a running container
<code>docker rm <container_id></code>	→ Remove a container

6. QUICK TIP



- One container = One service
- Keep images small
- Don't run too many processes in a single container



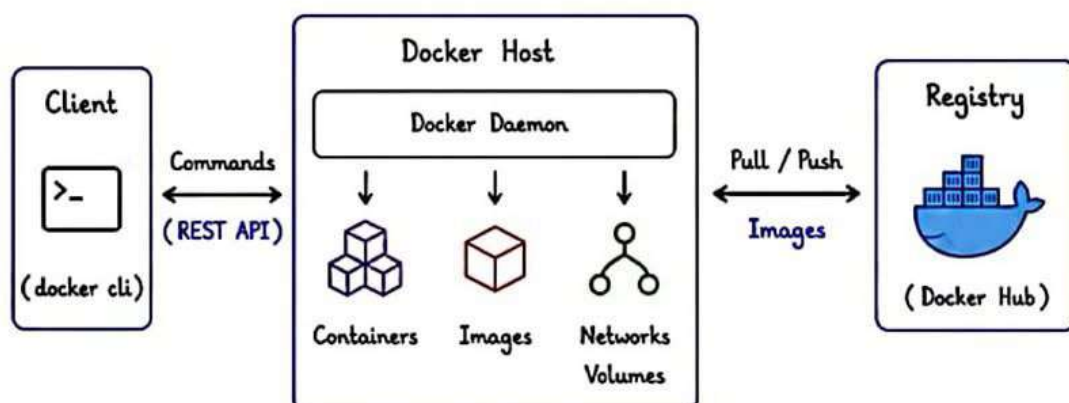
Remember: Build once, Run anywhere!



7. DOCKER ARCHITECTURE

Docker uses a **client-server** architecture.

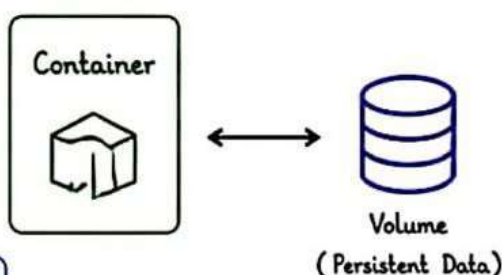
The Docker daemon manages images, containers, networks and volumes.



★ Docker Client sends commands → Docker Daemon executes → Manages everything.

8. VOLUMES

Volumes provide **persistent storage** for containers. Data remains even if container is removed.



Common Command

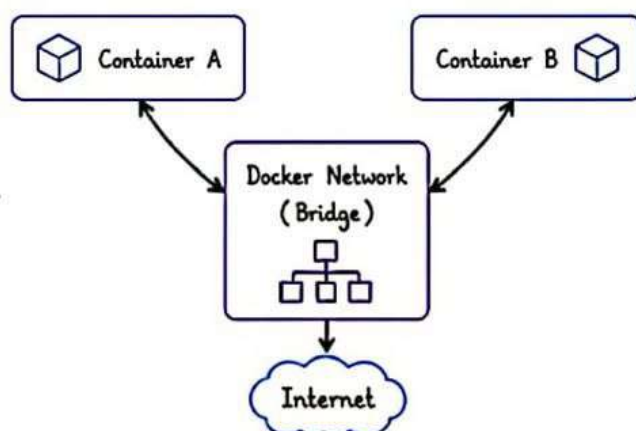
```
docker run -v <volume_name>:<path> image
```

Benefits

- ✓ Data persistence
- ✓ Share data between containers
- ✓ Backup easily
- ✓ Better performance

9. NETWORKING

Docker provides networking to allow containers to **communicate** with each other and the outside world.



Network Drivers

- bridge (default)
- host
- none
- overlay (for multiple hosts)
- macvlan



QUICK TIP

- Use `.dockerignore` to keep images small.
- Remove unused images and containers regularly.
- Understand volumes & networks well - very important!

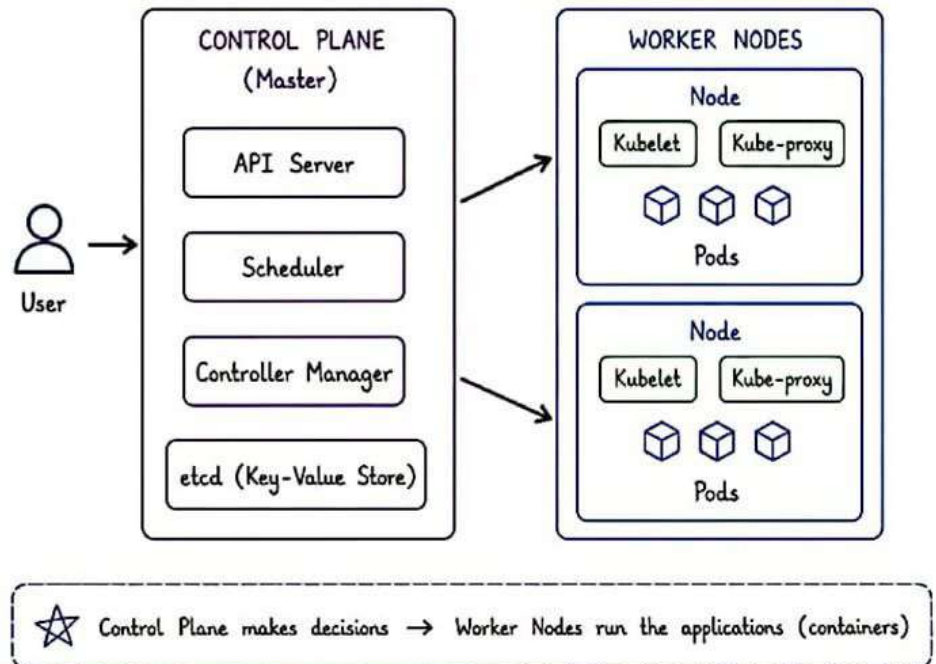


10. KUBERNETES (OVERVIEW)

Kubernetes (K8s) is an open-source platform to automate deployment, scaling and management of containerized applications.

💡 Why Kubernetes?

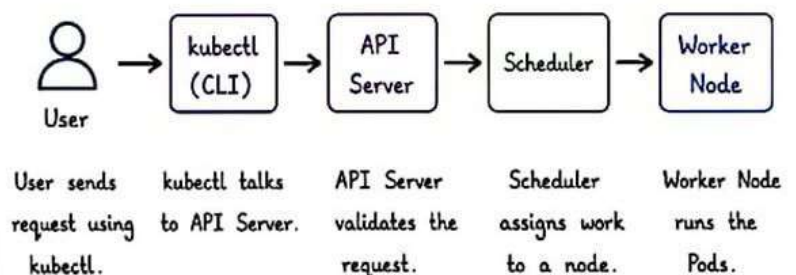
- ✓ Automated Deployment
- ✓ Scaling (Up & Down)
- ✓ Self-healing
- ✓ Load Balancing
- ✓ Efficient Resource Usage



12. KUBERNETES COMPONENTS (KEY)

- **Pod** : Smallest deployable unit. Runs one or more containers.
- **Service** : Provides stable IP & load balancing for Pods.
- **Deployment**: Manages ReplicaSets and handles updates & rollbacks.
- **ConfigMap** : Store configuration data as key-value pairs.
- **Secret** : Store sensitive information securely.
- **Namespace** : Logical isolation in a cluster.

13. HOW KUBERNETES WORKS (FLOW)



Example:

`kubectl apply -f app.yaml` → Creates/Updates resources in K8s

14. KUBECTL – COMMON COMMANDS

<code>kubectl get pods</code>	List all pods
<code>kubectl get nodes</code>	List all nodes
<code>kubectl get svc</code>	List all services
<code>kubectl apply -f <file.yaml></code>	Create / Update resources
<code>kubectl describe <resource> <name></code>	Show details of a resource
<code>kubectl delete -f <file.yaml></code>	Delete resources

💡 Remember:

- Kubernetes manages containers at scale.
- You describe the desired state, K8s makes it happen.
- Everything in K8s is a resource.

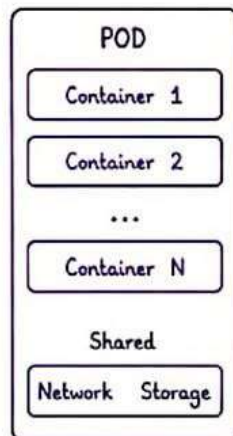


Kubernetes = Automation + Scalability + Reliability for your containers!



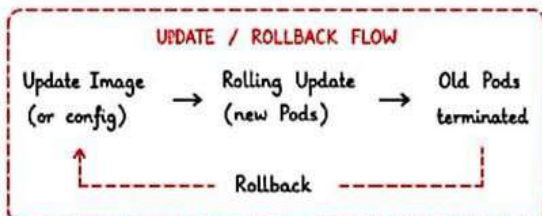
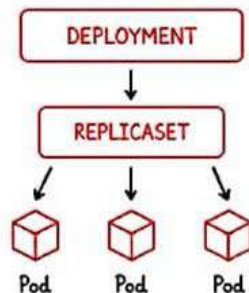
15. PODS (THE SMALLEST UNIT)

- Pod is the smallest deployable unit in Kubernetes.
- A Pod can contain one or more containers.
- Containers in a Pod share the same network & storage.
- Pods are ephemeral (can be created / destroyed).



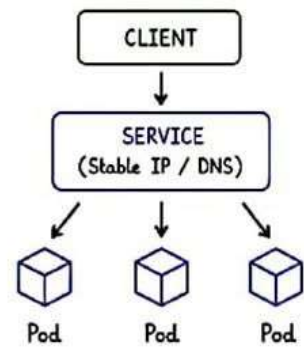
17. DEPLOYMENT

- Deployment manages ReplicaSets and provides declarative updates.
- Handles scaling, rollbacks and self-healing.
- Ensures the desired number of Pod replicas are running.



16. SERVICES

- Service provides a stable IP and DNS name to access Pods.
- It acts as a load balancer across multiple Pod instances.
- If Pods change, Service keeps working.

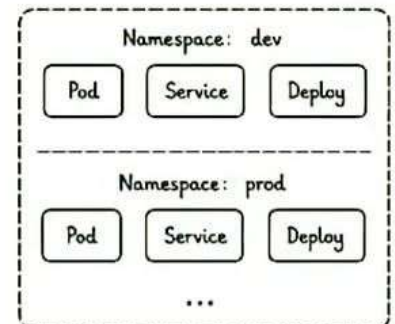


SERVICE TYPES

- ClusterIP - Internal access (default)
- NodePort - Access from outside (NodeIP:Port)
- LoadBalancer - External load balancer (Cloud)
- ExternalName - Map to external DNS name

18. NAMESPACE

- Namespace provides logical isolation in a cluster.
- Useful for separating environments (dev, test, prod).
- Resources in one namespace are not visible in another.



19. SECRET & CONFIGMAP

- Secret → Store sensitive data (passwords, tokens, keys)
- ConfigMap → Store non-sensitive configuration data
- Both help keep app config separate from code/image.



20. INTERVIEW QUESTIONS (REAL QUESTIONS)

DOCKER - BASIC

- What is Docker?
- What is the difference between Image and Container?
- How does Docker build an image?
- What is a Dockerfile?
- How do you persist data in Docker?
- What is Docker networking?
- What is the difference between CMD and ENTRYPOINT?

KUBERNETES - BASIC

- What is Kubernetes?
- What are the key components of Kubernetes?
- What is the difference between Pod and Container?
- What is a Namespace?
- What is a Service in Kubernetes?
- What are the types of Services?
- What is the use of Label and Selector?

DEPLOYMENT & SCALING

- What is a Deployment?
- How is Deployment different from ReplicaSet?
- How does rolling update work?
- How do you rollback to a previous version?
- How would you scale an application in Kubernetes?
- What is Horizontal Pod Autoscaler?

ADVANCED / REAL-WORLD

- How does Kubernetes achieve high availability?
- How does a Pod get an IP address?
- What happens when a Node fails?
- How does Kubernetes perform scheduling?
- How would you handle configuration in a 12-Factor app?
- How do you monitor a Kubernetes application?
- How would you troubleshoot an application not working in Kubernetes?

21. QUICK SUMMARY



Docker
Package application with dependencies into images and run as containers.



Kubernetes
Orchestrate containers at scale with automation.



Pods
Smallest deployable unit. Contains one or more containers.



Services
Provide stable access to Pods and load balancing.



Deployments
Manage updates, scaling and rollbacks.